Lesson 8 — Pulse Width Modulation (PWM)

ATmega328P / Arduino Uno | Source: lecture 08 slides (System 15 fader), Exercise 6 | Builds on Lessons 5 (ADC), 6 (motor drivers) and 7 (timers)

WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON

- Explain **why** we need PWM and why it is "analog-**like**", not true analog.
- Calculate **duty cycle**, **average voltage**, the **OCR value** for a duty cycle, and the **PWM frequency** / prescale.
- Explain how a timer, comparator and **waveform generator** make a PWM signal on an OC pin, in non-inverted and inverted mode.
- Tell the four timer modes apart (normal, CTC, fast PWM, phase correct PWM) and set them with the **WGM** and **COM** bits.
- Fade an LED (System 15) and drive a motor speed from a temperature (Exercise 6), in C and with `analogWrite`.

GIVEN TO YOU IN THE EXAM

The register sheet shows the bit positions of `TCCR0A/B`, `TIFR0`, `TIMSK0` and the pin sheet shows the **PWM pins marked ~** (3, 5, 6, 9, 10, 11) and the OC names. Check your printed copies. **The WGM and COM codes, the formulas and the reasoning below are "learn it".**

1. Why PWM?

So far you can read digital and analog sensors, and you can switch **digital actuators** fully ON or fully OFF (directly, or through a BJT, MOSFET or relay). But many jobs need something **in between**:

- **Dim** a light instead of only on/off.
- **Slow down** a motor: a line-following car must slow down before a bend or it topples; a quadcopter must reduce its 4 motors **gradually** to land, otherwise it crashes.

The CPU is digital: a pin is either **0 V or 5 V**. The ATmega328P has **no DAC** (digital-to-analog converter), so it cannot output 2.5 V. **PWM** solves this: it gets an **analog-like** output **by digital means**.

THE IDEA IN ONE SENTENCE

Toggle a digital pin ON/OFF **fast enough** that the attached device does not react to each pulse, but to the **percentage of time the output is HIGH in one cycle**.

The bulb experiment (lecturer's example): a filament bulb needs a short time to fully light and to fully go dark. Switch it on, and before it is fully bright switch it off; before it is fully dark switch it on again, very rapidly. The bulb settles at a brightness **in between**. Keep it ON for a larger part of each cycle and it is brighter (about 20% on → dim, 90% on → nearly full). The signal is still pure on/off (digital), but you *see* a smooth (analog-like) brightness.

2. Duty cycle and average voltage

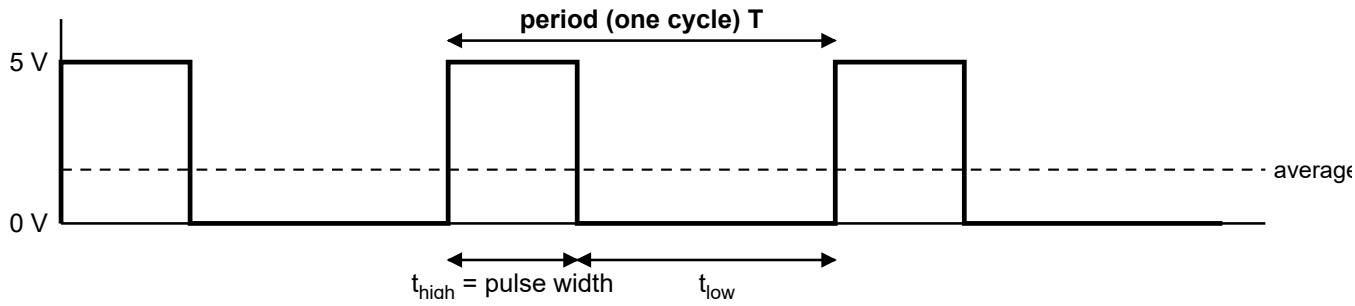


Figure 1. A PWM signal. Here t_{high} is one third of the period: duty cycle 33%, average 1.67 V.

FORMULAS

duty cycle = t_{high} / (t_{high} + t_{low}) × 100% = **pulse width / period × 100%**

average voltage = $\mathbf{V_{high}}$ **× duty cycle** (V_{high} = 5 V on the Uno)

For a fixed frequency the period is fixed (T = 1/f). So the only thing we change is the **width of the pulse**: changing (modulating) the pulse width changes the duty cycle, the average voltage, and so the brightness or speed. That is the name: **pulse width modulation**.

Duty cycle	0%	25%	50%	75%	100%
Average (voltmeter reads)	0 V	1.25 V	2.5 V	3.75 V	5 V

Voltmeter vs oscilloscope: a multimeter on a 50% PWM pin shows **2.5 V**, because it averages, just like your eye or a motor. An **oscilloscope** (voltage against time) shows the truth: a square wave jumping between 0 and 5 V. PWM is an "illusion" of analog.

3. How fast? Choosing the PWM frequency

- **Too slow**: the device follows each pulse. An LED **flickers**. For LEDs use **> 100 Hz**; in practice 500 Hz–1 kHz.
- **Too fast** for a slow device: a **motor** needs time to start, so it may **not start at all**. Toggle slower than the fastest change the device can follow.
- The right range comes from the device's **datasheet** (min and max supported PWM frequency). E.g. the datasheet says 100–300 Hz → set about 200 Hz.

4. How a timer makes PWM

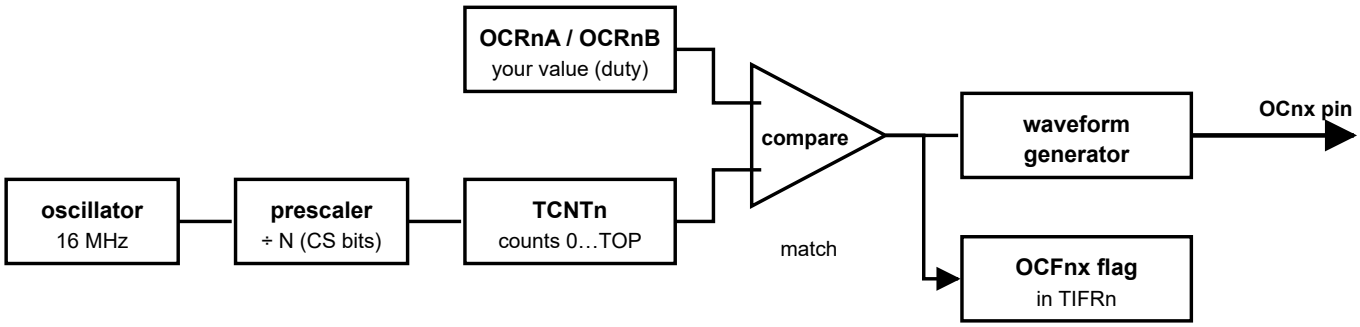


Figure 2. PWM hardware of one timer. The comparator compares TCNTn with OCRnx; on a match it sets the OCF flag and triggers the waveform generator, which drives the OC pin.

1. The timer counts `TCNTn` from 0 (**BOTTOM**) up to **TOP** (255 for 8-bit), then overflows to 0, over and over (Lesson 7).
2. You put a number between BOTTOM and TOP into `OCRnA` (**Output Compare Register**). This number sets the duty cycle.
3. The **comparator** constantly compares TCNTn with OCRnA. When they are equal (a **compare match**) it sets the **OCFnA** flag in TIFRn and triggers the **waveform generator**.
4. The waveform generator switches the **OCnA pin**. In **non-inverted** mode it **clears** the pin (LOW) on the match and **sets** it (HIGH) when TCNTn returns to BOTTOM.

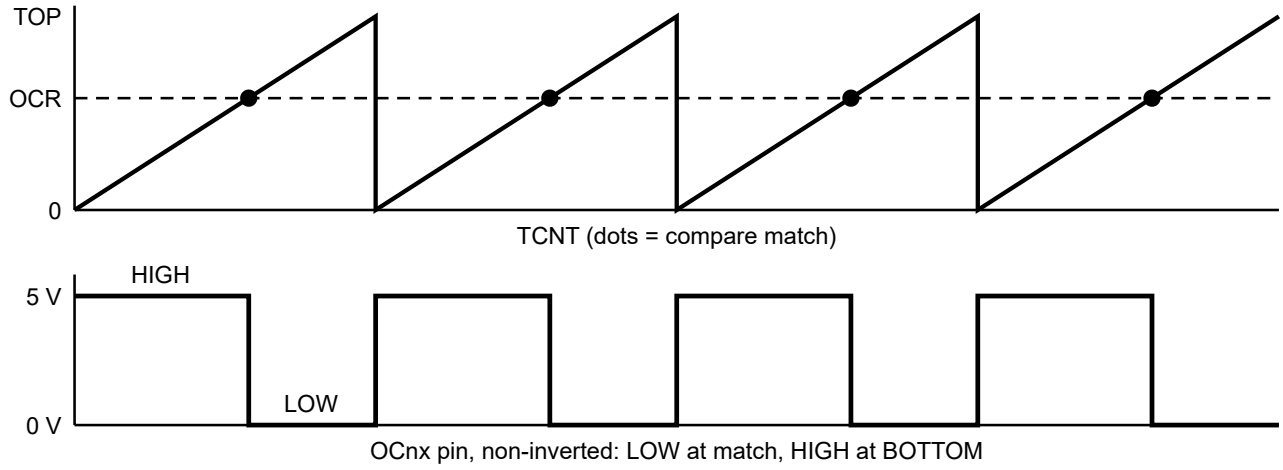


Figure 3. Fast PWM, non-inverted. Raise OCR → the match comes later → the pin stays HIGH longer → bigger duty cycle.

- **Inverted mode** is the opposite: **set** the pin on a match, **clear** it at BOTTOM. The duty cycle becomes (TOP − OCR) / (TOP + 1).
- Each timer has **one** TCNTn but **two** OCR registers (A and B), two comparators, two waveform generators and two OC pins. That gives **3 timers × 2 = 6 PWM pins**, marked **~** on the Uno.
- There are two control registers (TCCRnA and TCCRnB) simply because one 8-bit register cannot hold all the control bits.

DUTY CYCLE FROM OCR (FAST PWM, NON-INVERTED)

duty cycle = (OCR + 1) / (TOP + 1) × 100% ⇒ OCR = duty × (TOP + 1) − 1

8-bit: OCR 127 → 128/256 = **50%** (2.5 V); OCR 63 → **25%** (1.25 V); OCR 191 → **75%**; OCR 255 → 100% (5 V). OCR = 0 gives a very narrow spike (1/256), not a perfect 0 V. `analogWrite(pin, 0)` avoids this by simply writing the pin LOW.

5. PWM frequency and prescale

Counting from 0 up to TOP and back to 0 takes **TOP + 1** timer ticks, and that is **one PWM cycle**. So:

FAST PWM FREQUENCY

F_timer = F_CPU / prescale ⇒ **F_PWM = F_CPU / ((TOP + 1) × prescale)** ⇒ **prescale = F_CPU / ((TOP + 1) × F_PWM)**

TOP = $2^8 - 1 = 255$ for 8-bit, $2^{16} - 1$ for 16-bit. (Phase correct PWM counts up *and* down, so it is about **half** this frequency: F_CPU / (2 × TOP × prescale).)

8-bit, 16 MHz, fast PWM	**/1**	**/8**	**/64**	**/256**	**/1024**
F_PWM	62.5 kHz	7.8 kHz	**976.6 Hz**	244 Hz	61 Hz
Phase correct	31.4 kHz	3.9 kHz	**490 Hz**	122.5 Hz	30.6 Hz

Worked (slides): want 1 kHz → prescale = 16 000 000 / (256 × 1000) = 62.5. No such prescale exists, so take the nearest, **64** → actual F_PWM = 16 000 000 / (256 × 64) = **976.5 Hz**, close to 1 kHz. (These are also the Arduino defaults: pins 5 and 6 ≈ 980 Hz, the others ≈ 490 Hz.)

6. The four timer modes (WGM bits)

WGM02 WGM01 WGM00	Mode	What the counter does	PWM?
`0 0 0`	**Normal** (default)	0 → TOP, overflow to 0, sets TOV. The timer/counter of Lesson 7.	no
`0 0 1`	**Phase correct PWM**	0 → TOP, then **down** TOP → 0 (no overflow). Pin cleared on a match while counting up, set on a match while counting down.	yes, lower frequency
`0 1 0`	**CTC** (Clear Timer on Compare)	0 → OCRnA, then the timer **clears TCNTn to 0** by itself and sets OCFnA. TOP is programmable.	no
`0 1 1`	**Fast PWM**	0 → TOP, overflow to 0. Pin cleared on the match, set at BOTTOM (Figure 3).	yes, higher frequency

WGM01 and WGM00 are bits 1–0 of `TCCR0A`; WGM02 is bit 3 of `TCCR0B`. Because they all default to 0, the timers of Lesson 7 were already in **normal** mode without you touching them. (Timer1 has four WGM bits; the table above is for the 8-bit Timer0/Timer2.)

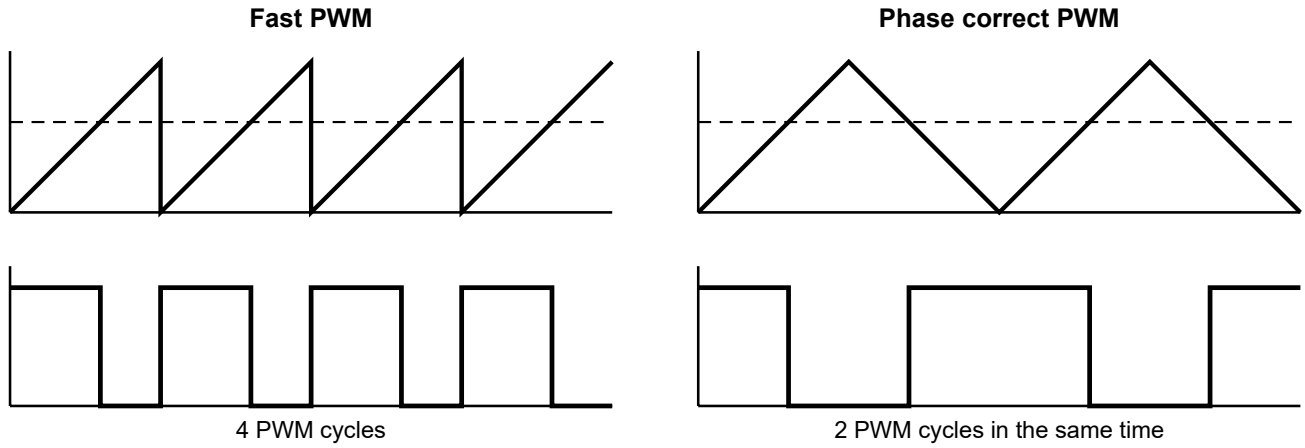


Figure 4. Same prescale, same OCR: fast PWM gives about twice as many cycles (higher frequency) as phase correct PWM.

WHICH PWM MODE FOR WHICH DEVICE

Without changing the prescale, **fast PWM** gives a **high** frequency, which suits **LEDs** (no flicker). **Phase correct** gives about half the frequency, so it is the traditional choice for **motors**, which need time to start.

7. The PWM registers bit by bit (Timer0)

`TCCR0A`: compare output mode (COM) and WGM01/00

	7	6	5	4	3	2	1	0	
TCCR0A	CA1	CA0	CB1	CB0	–	–	WG1	WG0	CA = COM0A, CB = COM0B, WG = WGM0

`TCCR0B`: WGM02 and the clock select bits (Lesson 7)

	·	·	–	–	WG2	CS2	CS1	CS0	
TCCR0B									011 = /64

COMnx1 COMnx0	OCnx pin (in a PWM mode)
`0 0`	disconnected: the pin is an ordinary I/O pin
`1 0`	**non-inverted**: clear on match, set at BOTTOM
`1 1`	**inverted**: set on match, clear at BOTTOM

Register	Bits for PWM / compare
`OCR0A`, `OCR0B`	value compared with TCNT0: 0–255 (16-bit timer: 0–65 535)
`TIFR0`	bit 2 OCF0B, bit 1 **OCF0A** (compare match flags), bit 0 TOV0. All flags: **write 1 to clear**.
`TIMSK0`	bit 2 OCIE0B, bit 1 **OCIE0A** (compare match interrupt enable), bit 0 TOIE0

Uno pin	3	5	6	9	10	11
AVR pin / OC name	PD3 / OC2B	PD5 / OC0B	**PD6 / OC0A**	PB1 / OC1A	PB2 / OC1B	PB3 / OC2A
Timer	2 (8-bit)	0 (8-bit)	0 (8-bit)	1 (16-bit)	1 (16-bit)	2 (8-bit)

8. System 15: the fader (LED fades from full to off, again and again)

Hardware: LED + 220 Ω on **D6 (PD6 = OC0A)** to GND. It **must** be a ~ pin: an LED on D8 cannot be dimmed by PWM. **Settings:** fast PWM (LED, high frequency), non-inverted, prescale 64 → 976.5 Hz.

```
// faderC.c
#include <avr/io.h>
#define F_CPU 16000000UL                    // CPU clock for delay.h
#include <util/delay.h>                     // _delay_ms()

int main(void)
{
    DDRD = DDRD | 0b01000000;                       // PD6 (OC0A) output
    TCCR0B = TCCR0B & 0b11110111;                   // WGM02 = 0
    TCCR0A = TCCR0A | 0b00000011;                   // WGM01 = 1, WGM00 = 1: fast PWM
    TCCR0A = (TCCR0A & 0b10111111) | 0b10000000;    // COM0A1 = 1, COM0A0 = 0: non-inverted
    TCCR0B = (TCCR0B & 0b11111011) | 0b00000011;    // CS = 011: /64 -> ~1 kHz
    while (1)
    {
        for (int pulseWidth = 255; pulseWidth >= 0; pulseWidth -= 5)
        {
            OCR0A = pulseWidth;                     // new duty cycle
            _delay_ms(30);                          // so the eye can see the fade
        }
    }
    return 0;
}
```

LABELED-CONSTANT VERSION (SAME EFFECT)

`DDRD |= (1<<DDD6);`
`TCCR0B &= ~(1<<WGM02);`
`TCCR0A |= (1<<WGM01) | (1<<WGM00);`
`TCCR0A = (TCCR0A & ~(1<<COM0A0)) | (1<<COM0A1);`
`TCCR0B = (TCCR0B & ~(1<<CS02)) | (1<<CS01) | (1<<CS00);`

ARDUINO VERSION

`int ledPin = 6;`
`setup: pinMode(ledPin, OUTPUT);`
`loop: for (int w = 255; w >= 0; w -= 5) { analogWrite(ledPin, w); delay(30); }`
`analogWrite` sets mode, COM and frequency behind the scenes; you only give the pin and the width (0–255).

OCR 255 → 5 V, about 125 → 2.5 V, about 63 → 1.25 V, down to 0. Without the 30 ms delay the whole fade would take under a millisecond and you would not see it.

9. True analog output

PWM is fine for lights and motors, but for **audio or music** a real continuously varying voltage gives much better quality. Three options:

1. **A microcontroller with a built-in DAC**, e.g. the AT91SAM3X8E on the **Arduino Due** (10-bit DAC: 0–1023). Expensive.
2. **A DAC breakout board**: cheap, but uses several I/O pins to talk to it.
3. **An RC filter** (resistor + capacitor) on the PWM pin smooths the pulses into an analog voltage. R and C depend on the PWM frequency (formula beyond this course).

10. Build example: Exercise 6 — the auto fan

Task: an LM35 measures room temperature. **≤ 25 °C → motor OFF**, **≥ 35 °C → full speed**, and in between the speed rises and falls **smoothly**. This one system uses everything: a **sensor** (Lesson 6), the **ADC** (Lesson 5), **PWM** (this lesson) and a **transistor driver** (Lesson 6).

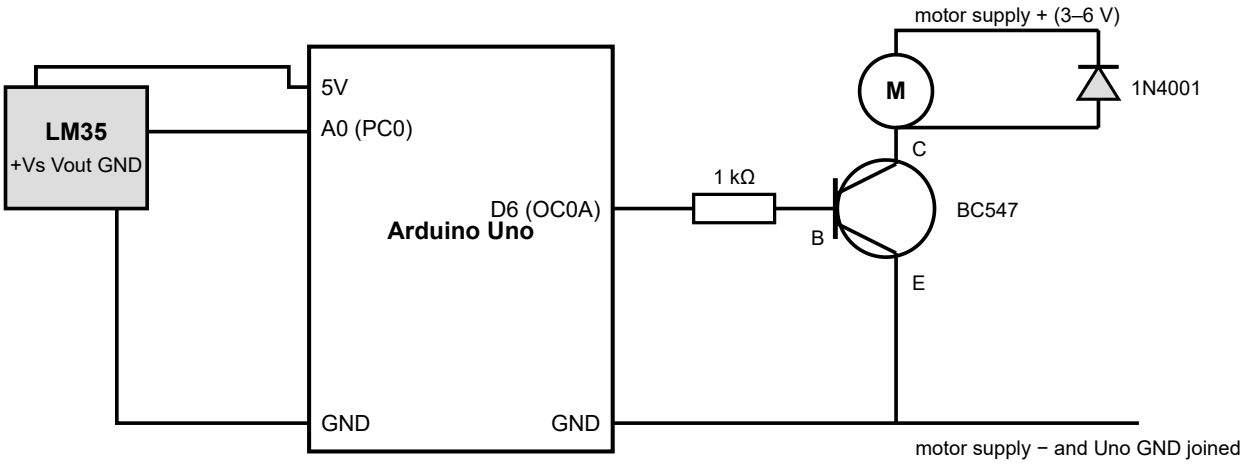


Figure 5. Auto fan. LM35 (flat face toward you: +Vs, Vout, GND) on A0. D6 drives the BC547 base through 1 kΩ; motor between its own supply + and the collector; flyback diode across the motor (stripe to +).

Why these choices: D6 is a PWM pin. The motor can't run from the pin (Lesson 6), so a transistor switches it. The motor has its **own supply** with the grounds joined. Check the motor current: the BC547 handles at most 100 mA; for a bigger motor use a 2N7000 (200 mA) or a power transistor. For a motor, **phase correct PWM** (≈ 490 Hz at /64) is the traditional choice.

The mapping. LM35: 10 mV/°C. With Vref = 5 V: T(°C) = ADC × 500 / 1024. Then:

T ≤ 25 → OCR = 0 (off) | T ≥ 35 → OCR = 255 (full) | between: **OCR = (T − 25) × 255 / 10** (e.g. 30 °C → 127 ≈ 50%)
Here Vref = 5 V keeps the maths simple (1 step ≈ 0.49 °C). The Part 1 notes use the **1.1 V internal reference** instead (ADMUX REFS = 11, T = ADC × 110 / 1024), which gives finer steps (≈ 0.11 °C) for the 0–1 V the LM35 produces in a room. Either is acceptable.

```
// Exercise 6 Q3: auto fan in C (no wrapper functions)
#include <avr/io.h>

int main(void)
{
    DDRD = DDRD | 0b01000000;                       // PD6 (OC0A) output: motor driver
    TCCR0A = (TCCR0A & 0b11111100) | 0b00000001;    // WGM01 = 0, WGM00 = 1: phase correct PWM
    TCCR0B = TCCR0B & 0b11110111;                   // WGM02 = 0
    TCCR0A = (TCCR0A & 0b10111111) | 0b10000000;    // COM0A = 10: non-inverted
    TCCR0B = (TCCR0B & 0b11111011) | 0b00000011;    // /64 -> ~490 Hz
    ADMUX = 0b01000000;                             // Vref = AVcc (5 V), right-adjusted, channel ADC0
    ADCSRA = 0b10000111;                            // ADC enable, prescale /128

    while (1)
    {
        ADCSRA = ADCSRA | 0b01000000;               // ADSC = 1: start conversion
        while (ADCSRA & 0b01000000) { }             // wait until ADSC = 0 (done)
        long t10 = (long)ADCW * 5000 / 1024;        // temperature x10 (e.g. 302 = 30.2 C)

        if (t10 <= 250)
            OCR0A = 0;                              // 25 C or lower: off
        else if (t10 >= 350)
            OCR0A = 255;                            // 35 C or higher: full speed
        else
            OCR0A = (t10 - 250) * 255 / 100;        // smooth in between
    }
}
```

Q2: ARDUINO VERSION

```
int fan = 6;
setup: pinMode(fan, OUTPUT);
loop:
 float t = analogRead(A0) * 500.0 / 1024;
 int w = (t - 25) * 255 / 10;
 analogWrite(fan, constrain(w, 0, 255));
```

(`constrain` clips below 25 °C to 0 and above 35 °C to 255.)

PRACTICAL TIPS

Test the PWM with an **LED** on D6 first (it should brighten as you warm the LM35 with your fingers), then connect the transistor and motor. Warm the LM35 by holding it; cool it by blowing. A motor may not start at a small duty cycle: that is normal (it needs enough average voltage to overcome friction).

L. What the lecturer explained in the lectures (2020 recordings)

From the three PWM recordings (folders 22–24 in `Embedded Systems Lectures`). **[EXAM]** marks things he tied to the exam.

L1. "ANALOG-LIKE", NOT ANALOG [EXAM]

He stressed the word **like**: "PWM is not true analog. It gives analog-like output." It is a digital technique, a kind of illusion. Users see and feel analog, a voltmeter even reads 2.5 V, but "only the oscilloscope can tell us the truth": it is a 0/5 V square wave. "You don't need any rocket science to understand PWM" (the bulb example), even though books are full of mathematics.

L2. Why PWM: his examples

- So far you can turn actuators fully on or off. PWM is how a **digital computer drives analog actuators**: dim a light, slow a motor.
- A small **line-following car** must slow down for a bend or junction or it topples, then speed up again. A **quadcopter** ("quad = four propellers") must take off and land gradually; switching the motors off is a crash landing.
- The filament bulb experiment (section 1): switch on and off faster than the filament can fully heat or cool. 20% on → dim, 40% → brighter, 90% → nearly full.

L3. Frequency and datasheets

- "Faster than a human eye can see any flicker": for LEDs **> 100 Hz**, normally **500 Hz to 1 kHz**.
- Slow devices like **motors** take time to start; toggle too fast and "the motor would not start at all". Toggle slower than the maximum speed the device can react.
- Find the supported PWM range in the **datasheet**. Every reliable component has one (even a resistor): dimensions, voltage, current, heat, PWM range, hazardous materials such as mercury or cadmium.
- Cheap local parts often come with no datasheet. Find the **identical clone on SparkFun** and download its datasheet ("get the cheap device from Pettah, the datasheet from SparkFun").

L4. Instruments

- A **multimeter** must at least be a voltmeter, ammeter and ohmmeter; modern ones also measure capacitance, inductance and frequency. A good (Japanese) one costs under Rs 10 000; cheap ones about Rs 1 000 are less accurate.
- An **oscilloscope** plots voltage against time. Even a cheap one costs Rs 40–50 000; a good one can be about Rs 500 000.
- His quiz: 50% duty → 2.5 V; for 1.25 V use **25%**; 75% → **3.75 V**.

L5. How the waveform is made [EXAM]

- "To use PWM you don't need to know all these things", but as university students you must know **how the waveform generator works**. He said many good embedded programmers know how to use PWM but not this. At graduation you should be able to **self-learn** what wasn't covered ("we stopped at S; T to Z you learn yourself").
- Duty cycle formula he wrote: **(OCR + 1) × 100 / (TOP + 1)**. OCR 127 → 50%, OCR 63 → 25%.
- OCR at TOP keeps the pin at 5 V; near BOTTOM it is (almost) always 0 V. OCR closer to TOP = longer HIGH time.
- Derivation of F_PWM: with TOP = 3 the counter makes 0→1→2→3→0 = **4 = TOP + 1** ticks per PWM cycle. So $t_{timer} = t_{PWM}$ / (TOP + 1), F_PWM = F_timer / (TOP + 1) = F_CPU / ((TOP + 1) × prescale).
- Student asked why there are two control registers (TCCRnA and TCCRnB): there are too many control bits for one 8-bit register. Other peripherals (like the USART) do the same.

L6. Modes

- Normal mode is why the Lesson 7 timers worked without touching WGM: all WGM bits default to 0.
- **CTC** example: count to 5 by putting 5 in OCR and watching OCF; no need to reset TCNT, because the timer clears it itself. A student asked why CTC exists when normal mode can do the same. He said honestly he would check and let them know, and that CTC "is not widely used".
- He prompted the class to ask "why two PWM modes?" and answered: for the same prescale, **fast PWM gives more pulses (higher frequency) for LEDs**, while **phase correct gives fewer (about half, e.g. 1 kHz vs 500 Hz) for motors**. You get both just by changing the mode.
- **Dimming a 230 V bulb?** Not with a relay: a relay cannot switch on and off fast enough for PWM. Use a **TRIAC** (3 pins: gate, T1, T2). He **discouraged** it because of the shock risk with mains, unless you have experience.

L7. Fader details [EXAM]

- A common mistake: **connecting the device to a pin that is not PWM-capable** (e.g. D8) and expecting a fade. Use a ~ pin; he used D6 = OC0A of Timer0.
- Fast PWM for the LED (avoid flicker), non-inverted ("the widely used mode"), 1 kHz.
- Prescale for 1 kHz = 62.5, which does not exist, so use **64** → 976.5 Hz, "not exactly 1 kHz".
- The 30 ms delay is there only so the eye can see the fade.
- Demo with `analogWrite(9, 250)`: the multimeter read **4.72 V**, not 5 V (250/255 × 5 ≈ 4.9 V; a real pin and meter read a little lower).
- The Arduino version hides WGM, COM and frequency inside `analogWrite`. You only give the pin and the width.

L8. True analog, and the reaction-timer seed

- True analog matters for quality **audio/music**. Three options: a built-in DAC (**Arduino Due**, AT91SAM3X8E, 10-bit DAC; about Rs 4 000 for a clone, Rs 12 000 original), a **DAC breakout** (about Rs 300–350, but it uses about 4 I/O pins), or an **RC filter** (formula is beyond the course).
- Exercise 5 question: why seed `srand`? Without a random seed, anyone who knows the algorithm can predict the numbers. PCs use the system clock; a microcontroller has none. So use the **elapsed time from power-up to the first press** (a timer), or a **floating analog pin**, which acts like an **antenna** and picks up random electromagnetic noise.
- He prefers to **guide** students to answers rather than give them. Keep asking questions.
- Next lesson: data communication (serial, parallel, synchronous, asynchronous), first what happens behind `Serial.print`, then Arduino.

11. Common mistakes

Mistake	What happens / fix
Device on a non-PWM pin	No fading, only on/off. Use 3, 5, 6, 9, 10 or 11 (~).
COM bits left 00	Timer runs but the pin is not connected to the waveform generator. Set COM0A1 = 1.
Forgetting DDR for the OC pin	No output. The OC pin must be an output.
Clock select left 000	Timer stopped, no PWM.
Wrong WGM split	WGM01/00 are in TCCR0A, WGM02 in TCCR0B.
Expecting exactly 1 kHz	Only 1, 8, 64, 256, 1024 exist: /64 gives 976.5 Hz.
Motor straight on the pin	Too much current. Use a transistor, a flyback diode and its own supply.
PWM too fast for a motor / too slow for an LED	Motor won't start / LED flickers. Pick the frequency from the datasheet.
Inverted when you wanted non-inverted	Brightness goes the opposite way. COM = 10 non-inverted, 11 inverted.

Cheat sheet

Formulas	
Duty	t_{high} / period × 100%
Average V	5 V × duty
Duty from OCR	(OCR + 1) / (TOP + 1)
Fast PWM f	F_CPU / ((TOP+1) × N)
Prescale	F_CPU / ((TOP+1) × f)
Phase correct	≈ half of fast PWM
8-bit /64	976.5 Hz fast, 490 Hz phase correct

Registers (Timer0)	
WGM02:00	000 normal, 001 phase correct, 010 CTC, 011 fast PWM
COM0A1:0	10 non-inverted, 11 inverted (TCCR0A bits 7:6)
OCR0A	duty value 0–255 → pin D6
TIFR0 / TIMSK0	OCF0A / OCIE0A bit 1, OCF0B / OCIE0B bit 2
PWM pins	T0: D6 (A), D5 (B); T1: D9, D10; T2: D11, D3
Arduino	`analogWrite(pin, 0–255)`